1.Summary

This House Price Prediction machine learning project. The project follows a complete, industry-standard regression workflow: data understanding, data quality assessment, exploratory data analysis (EDA), data cleaning, feature engineering, pipeline-based preprocessing, multi-model training, model evaluation and comparison, hyperparameter tuning, final model selection, and deployment preparation.

2. Problem Statement

Real estate prices are influenced by many interacting factors — size, location, age, amenities, and market conditions. Manually estimating a fair price for a property is time-consuming and inconsistent. The goal of this project is to build a machine learning model that can predict a property's price (house_price_inr) from its measurable characteristics, such as area, number of bedrooms and bathrooms, location, property age, and furnishing status.

Problem type: Supervised Regression — the target variable, house_price_inr, is a continuous numeric value.

Business value: A reliable price-prediction model can support fairer pricing for buyers and sellers, assist real estate platforms in flagging mispriced listings, and speed up property valuation.

3. Dataset Overview

The dataset contains 10,000 property records across 17 columns, combining numeric and categorical property attributes.

Key columns

- property_id — unique identifier for each property
- area_sqft — built-up area in square feet
- bedrooms, bathrooms — room counts
- property_age — age of the property in years
- location — city where the property is located
- location_score — a numeric desirability score for the location
- distance_city_km — distance from the city center
- crime_rate_index — a safety/crime indicator for the area
- property_type — Apartment, Villa, etc.
- furnishing_status — Furnished / Semi-Furnished / Unfurnished
- parking_spaces, floor_number, total_floors — building attributes
- has_garage, has_pool — amenity flags
- house_price_inr — target variable: property price in Indian Rupees

4. Project Workflow

The notebook follows this end-to-end pipeline:

- Environment setup and library imports
- Data loading and structural inspection
- Data quality assessment (missing values, duplicates)
- Exploratory Data Analysis (distributions, correlations, relationships)
- Outlier analysis
- Feature engineering
- Train/test split
- Preprocessing pipeline (imputation, scaling, encoding)
- Model training: Linear, Ridge, Lasso, Random Forest, XGBoost
- Model evaluation and comparison
- Cross-validation and hyperparameter tuning setup
- Final model selection, saving, and sample prediction

5. Environment Setup & Libraries

Importing Libraries

Before any analysis can happen, the required Python libraries must be imported.

Code

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
```

Advantages:

- Centralizes all dependencies at the top of the notebook, making requirements clear at a glance.
- Pandas and NumPy are the de-facto standard for tabular/numeric work in Python, ensuring compatibility with scikit-learn and XGBoost.
- Seaborn builds on Matplotlib to produce statistically informative plots (e.g., KDE curves, boxplots) with far less code.

Interpretation:

This cell establishes the technical foundation for the entire notebook. Pandas is used because the project is fundamentally about tabular data — rows of properties and columns of attributes — and Pandas' DataFrame is the standard structure for loading, filtering, grouping, and transforming this kind of data efficiently. NumPy is imported alongside it because several later steps (the log transformation of price, array-based calculations for RMSE, and outlier bounds) rely on NumPy's fast numerical routines rather than plain Python loops.

Matplotlib and Seaborn are imported together because Seaborn is built on top of Matplotlib: Seaborn provides high-level, statistically aware chart types (histograms with density curves, boxplots, heatmaps) with very little code, while Matplotlib is still used underneath for figure sizing, titles, and axis labels. Finally, train_test_split is imported at the very top even though it is not used until much later — this is a common professional practice that documents upfront that the project will follow a train/test evaluation methodology, signaling to any reader that the results reported later were measured on unseen data rather than the training data itself.

6. Data Loading & Initial Inspection

. Loading the Dataset

Code

```
df = pd.read_csv("/content/house_price_prediction_dataset_10000.csv")
df.head()
```

Advantages:

- Pandas' read_csv handles type inference, delimiters, and large files efficiently in one call.
- Previewing the data early catches column-naming or encoding issues before they propagate into later steps.

Interpretation:

This is the entry point of the entire project: the raw CSV file is read into a Pandas DataFrame named df, and every subsequent operation in the notebook builds on this object. Using read_csv() rather than manually parsing the file means Pandas automatically infers each column's data type (numbers vs. text), handles the header row, and produces a ready-to-use table in a single line of code.

Calling head() immediately afterward is a deliberate verification step rather than a formality — it confirms the file path was correct, the columns loaded with sensible names (property_id, area_sqft, bedrooms, location, etc.), and the values look reasonable (for example, area in the thousands of square feet, and bedroom counts in single digits). Catching a loading problem here, before any analysis is built on top of it, avoids far more expensive debugging later in the notebook.

6.2. Structural Inspection with info()

Code

```
df.info()
```

Advantages:

- Immediately reveals which columns need type conversion or missing-value handling.
- Avoids manually checking each column one at a time.

Interpretation:

info() is used here because it answers three important structural questions about the dataset at once: how many rows and columns exist, what data type each column holds, and how many non-null (i.e., present) values each column contains. This single call replaces what would otherwise require several separate inspections.

The output confirms the dataset has 10,000 rows and 17 columns, which matches the filename and expected scale of the project. More importantly, the Non-Null Count column shows that some columns — area_sqft, for example, with 9,980 non-null values instead of the full 10,000 — contain missing data. This is the first concrete evidence in the notebook that a missing-value handling strategy will be required before the data can be used to train a model, which directly motivates the Data Quality Assessment steps that follow in Section 7.

6.3. Checking Dataset Shape

Code

```
df.shape
```

```
OUTPUT
(10000, 17)
```

Advantages:

- A single, cheap operation that avoids scanning the full DataFrame just to count rows/columns.

Interpretation:

shape is used as a lightweight, unambiguous sanity check that returns exactly two numbers: the row count and the column count, without printing the entire dataset. It is typically re-checked at multiple points in a project (before and after cleaning, before and after feature engineering) to make sure operations like dropping columns or filtering rows behaved as expected.

The result, (10000, 17), confirms the dataset matches its expected size: 10,000 individual property records, each described by 17 attributes. This number becomes an important reference point later — for instance, when the train/test split is performed, the resulting 8,000/2,000 row split can be checked against this original total of 10,000 to confirm no rows were unintentionally lost or duplicated.

7. Data Quality Assessment

7.1. Counting Missing Values

Code

```
df.isnull().sum()
```

```
OUTPUT
area_sqft            20
location             15
location_score        15
crime_rate_index      10
property_type         12
furnishing_status     10
dtype: int64
```

Advantages:

- Vectorized operation — computed across the entire DataFrame in one pass rather than looping through rows.
- Directly informs which imputation strategy is needed for each column.

Interpretation:

isnull() flags every individual cell in the DataFrame as True (missing) or False (present), and chaining .sum() on top adds up the True values per column, producing a per-column count of missing entries in a single, efficient operation. This is far more practical than manually scrolling through the data or writing a loop to check each row.

The output pinpoints exactly which columns are affected — area_sqft (20 missing), location (15), location_score (15), crime_rate_index (10), property_type (12), and furnishing_status (10) — while every other column, including the target house_price_inr, has zero missing values. Because both numerical columns (area_sqft, location_score, crime_rate_index) and categorical columns (location, property_type, furnishing_status) are affected, this result confirms that two separate imputation strategies will be needed, one for each data type, which is exactly what the preprocessing pipelines in Section 12 are built to handle.

7.2. Visualizing Missing Values

Code

```
plt.figure(figsize=(12, 6))

sns.heatmap(
    df.isnull(),
    cbar=False,
    yticklabels=False
)

plt.title('Missing Values Heatmap')
plt.show()
```

Advantages:

- Visual pattern recognition is faster than reading a text summary, especially for larger datasets.
- Helps distinguish random missingness from systematic missingness, which affects the choice of imputation method.

Interpretation:

This heatmap plots df.isnull() directly, where every cell in the dataset is colored according to whether it is missing or present. This turns the numeric summary from the previous cell into an image that can be scanned visually across all 10,000 rows and 17 columns at once — something that would be impractical to do by reading a table of numbers.

The resulting pattern shows missing values as thin, scattered lines rather than large solid blocks. This tells us the missingness is spread fairly randomly across different rows and columns rather than being concentrated in, for example, a specific batch of records that might indicate a data collection or export error. This distinction matters practically: random, scattered missingness like this is generally safe to handle with simple imputation methods (as used in Section 12), whereas a systematic pattern would call for a more careful, column-specific investigation.

7.3. Missing Value Percentage

Code

```
missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_summary = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Missing Percentage": missing_percentage
})

missing_summary[missing_summary["Missing Values"] > 0]
```

OUTPUT

```
                Missing Values  Missing Percentage
area_sqft                   20                0.20
location                    15                0.15
location_score              15                0.15
```

Advantages:

- Puts every column on a common, comparable scale regardless of dataset size.
- Supports a data-driven decision rule (e.g., 'impute under 5%, consider dropping above 40%').

Interpretation:

Raw missing-value counts do not, by themselves, indicate severity — 20 missing values means something very different in a dataset of 100 rows than in a dataset of 10,000 rows. This step divides each column's missing count by the total row count and multiplies by 100 to express missingness as a percentage, which is a scale-independent way of judging how serious the gap really is.

The output shows that every affected column has well under 1% of its values missing (area_sqft at 0.20%, location and location_score at 0.15% each, and so on for the remaining columns). This is a small enough proportion that dropping rows or columns would discard useful data unnecessarily; instead, filling in these gaps with a sensible estimate (median for numeric columns, most frequent category for categorical columns, as implemented later in the preprocessing pipeline) is both safe and standard practice at this level of missingness.

7.4. Duplicate Record Check

Code

```
df.duplicated().sum()
```

OUTPUT

```
np.int64(0)
```

Advantages:

- A single vectorized call checks all 10,000 rows for exact duplication.
- Confirms the dataset does not need de-duplication, saving an unnecessary cleaning step.

Interpretation:

duplicated() compares every row in the DataFrame against every other row and flags rows that are exact repeats. Summing the result gives a single count of how many duplicate rows exist. This check matters because duplicate records, if present, would cause the model to see the same underlying observation multiple times, effectively giving it extra, artificial weight during training and skewing the learned relationships.

The result of 0 confirms there are no exact duplicate property records in this dataset, so no de-duplication step is required. This is a positive data-quality finding — it means the 10,000 rows represent 10,000 genuinely distinct observations, and the train/test split performed later can be trusted to produce two disjoint sets of unique properties rather than accidentally placing the same record in both the training and testing partitions.

7.5. Statistical Summary

Code

```
df.describe()

# For categorical columns:
df.describe(include="object")
```

OUTPUT

```
        area_sqft      bedrooms     bathrooms  ...
count  9980.000000  10000.000000  10000.000000
mean   2737.780661      2.869000      2.659000
```

```
std    1309.654976      1.005555      1.002706
...
```

Advantages:

- Surfaces impossible or extreme values (e.g., negative areas) without manual inspection.
- Gives an at-a-glance sense of scale differences between features, which motivates the scaling step used later.

Interpretation:

describe() computes count, mean, standard deviation, minimum, quartiles, and maximum for every numeric column, and describe(include='object') computes count, number of unique values, most frequent value, and its frequency for every categorical column. Together they give a fast, comprehensive statistical profile of the dataset without needing to inspect each column individually.

For the numeric summary, the average property size (mean area_sqft of roughly 2,738 sq ft) and the standard deviation give a sense of how varied the properties are, and the count column (9,980 for area_sqft versus 10,000 for bedrooms) is a second confirmation of the missing-value pattern already identified. For the categorical summary, the output shows that Chennai is the most frequent location and Apartment is the most common property type, which is useful context when later interpreting model behavior and feature importance — for example, understanding that the model has seen many more Apartment listings than other types.

8. Exploratory Data Analysis (EDA)

8.1. Target Variable Distribution

Code

```
sns.histplot(data=df, x="house_price_inr", kde=True)
```

Advantages:

- A histogram with a KDE overlay shows both the discrete shape and the smoothed trend in a single plot.
- Skew detected here justifies the log-transformation applied later — a data-driven decision rather than a default assumption.

Interpretation:

histplot with kde=True is used to visualize the shape of the target variable's distribution — the bars show how many properties fall into each price range, while the smooth KDE (Kernel Density Estimate) curve overlaid on top shows the overall trend without the noise of individual bin boundaries. Understanding the target's distribution is a critical early step because many regression algorithms, especially linear ones, are sensitive to how symmetric or skewed the value they are predicting is.

The plot shows house prices concentrated at lower-to-mid price ranges, with a long right tail caused by a smaller number of high-value properties. This visual impression is later confirmed numerically by a skewness value of approximately 1.39 (Section 8.2), and it directly motivates the log-transformation performed in Section 8.3 — without addressing this skew, the small number of very expensive properties would disproportionately influence error metrics like RMSE, since squared errors penalize large deviations heavily.

8.2. Boxplot & Skewness Check

Code

```
sns.boxplot(x=df["house_price_inr"])

target_skewness = df["house_price_inr"].skew()
```

OUTPUT

```
House Price Skewness: 1.3856512390309388
```

Advantages:

- Boxplots make outliers immediately visible as isolated points beyond the whiskers.
- A single skewness statistic gives an objective, reproducible measure that can be tracked before/after transformation.

Interpretation:

The boxplot summarizes the price distribution using its median, quartiles, and whiskers, and plots any values beyond 1.5 times the interquartile range as individual points — these points represent unusually high-priced properties relative to the rest of the dataset. Pairing this visual check with the numeric skew() calculation avoids relying on visual judgment alone, since 'how skewed' something looks can be subjective from a chart.

The calculated skewness of 1.386 is a positive value well above the 0.5–1.0 range typically considered 'moderately skewed,' confirming numerically what the histogram suggested: house prices are meaningfully right-skewed, driven by a minority of high-value properties pulling the distribution's tail outward. This single number becomes a useful before/after benchmark — it is compared again after the log transformation in the next step to quantify how much that transformation actually improved the distribution's symmetry.

8.3. Log-Transforming the Target

Code

```
df["log_house_price"] = np.log1p(df["house_price_inr"])

sns.histplot(data=df, x="log_house_price", kde=True)
```

Advantages:

- Many regression models (especially linear ones) perform better and produce more stable error estimates on a roughly symmetric target.
- log1p (rather than plain log) safely handles values at/near zero without producing errors or -infinity.

Interpretation:

np.log1p() computes log(1 + x), which compresses large values far more aggressively than small ones. Applied to house prices, this means an expensive property's price is 'pulled in' toward the bulk of the distribution much more than a cheap property's price is, which is precisely the effect needed to counteract the right-skew identified in the previous step. The '+1' in log1p (rather than a plain logarithm) is a safeguard that keeps the calculation well-defined even if a price were ever zero, avoiding an undefined or negative-infinity result.

The resulting histogram of log_house_price is visibly more symmetric and bell-shaped than the original price distribution, and this new column becomes the actual prediction target used throughout the rest of the notebook (y_log in Section 11). This is a deliberate modeling decision: training on a more symmetric target tends to produce more stable, better-calibrated models, and predictions are converted back to real INR values later using the inverse transformation, np.expm1(), whenever results need to be reported or interpreted in rupee terms.

8.4. Feature Distributions & Correlation

Code

```
df[numerical_columns].hist(figsize=(18, 14), bins=30)

correlation_matrix = df[numerical_columns].corr()
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f")

price_correlation =
df[numerical_columns].corr()["house_price_inr"].sort_values(ascending=False)
```

OUTPUT

```
house_price_inr     1.000000
area_sqft           0.702199
location_score       0.119380
bedrooms             0.053611
...
```

Advantages:

- A single heatmap replaces dozens of individual pairwise comparisons.
- Sorting correlations against the target directly ranks candidate predictive features.
- Reveals redundant or weak features early, informing feature selection decisions.

Interpretation:

The histogram grid gives a quick visual overview of every numeric column's distribution at once — useful for spotting which features are roughly normal, which are skewed, and which are more discrete (like bedroom or bathroom counts). The correlation heatmap goes a step further by quantifying the linear relationship between every pair of numeric columns on a scale from -1 to +1, with the color scale making strong positive or negative relationships visually obvious without reading every individual number.

Sorting the correlations against house_price_inr specifically produces a ranked list of which features are most linearly associated with price. The standout result is area_sqft, with a correlation of about 0.70 — a strong, well-known real estate relationship (bigger properties cost more). By contrast, most other original features, including bedrooms

(0.054), bathrooms (0.050), and location_score (0.119), show only weak linear correlation with price individually. This is an important, honest finding: it suggests that price in this dataset is driven substantially by size and less by these particular secondary attributes on their own — though weak individual correlation does not rule out that a feature contributes through interactions that tree-based models like Random Forest and XGBoost can capture but a correlation coefficient cannot.

8.5. Categorical & Relationship Plots

Code

```
sns.countplot(data=df, x="location", order=df["location"].value_counts().index)
sns.boxplot(data=df, x="location", y="house_price_inr")
sns.boxplot(data=df, x="property_type", y="house_price_inr")
sns.scatterplot(data=df, x="area_sqft", y="house_price_inr")
```

Advantages:

- Count plots catch class imbalance in categorical features before one-hot encoding.
- Grouped boxplots make it easy to see which locations or property types tend to command higher prices.
- Scatter plots visually confirm relationships already suggested by the correlation matrix (e.g., area vs. price).

Interpretation:

This group of plots examines relationships that a simple numeric correlation matrix cannot capture, since correlation only applies to numeric variables and location/property_type are categorical. countplot shows how many listings exist per category (useful for spotting imbalance before one-hot encoding, which is applied later in Section 12.2), while grouped boxplot compares the full price distribution — not just the average — across categories such as location or property type. The scatterplot for area vs. price gives a direct, point-by-point view of that specific relationship rather than relying only on the summary correlation coefficient.

The location count plot shows some cities (such as Chennai) have noticeably more listings than others, meaning the dataset is not perfectly balanced across locations — something to keep in mind when interpreting how confidently the model can predict prices in less-represented cities. The location and property-type boxplots show visibly different median prices and price spreads across categories, indicating that both location and property type carry real pricing signal even though they were not part of the earlier numeric correlation analysis. The area-vs-price scatterplot shows a clear upward trend consistent with the 0.70 correlation found earlier, visually confirming that relationship at the level of individual properties rather than just as a summary statistic.

9. Outlier Analysis

9.1. IQR-Based Outlier Detection

Code

```
Q1 = df[numerical_columns].quantile(0.25)
Q3 = df[numerical_columns].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outlier_count = (
    (df[numerical_columns] < lower_bound) |
    (df[numerical_columns] > upper_bound)
).sum()
```

OUTPUT

```
bathrooms           400
house_price_inr     288
log_house_price      63
...
```

Advantages:

- Works consistently across features with very different units and scales.
- Provides an objective, threshold-based outlier count rather than relying on visual judgment alone.
- Directly demonstrates the benefit of the log transform: log_house_price has far fewer outliers (63) than raw house_price_inr (288).

Interpretation:

This code implements the standard IQR (Interquartile Range) rule for outlier detection: it calculates the 25th and 75th percentiles for every numeric column, computes the spread between them (the IQR), and then flags any value falling more than 1.5 times that spread below Q1 or above Q3 as an outlier. This method is used instead of a fixed numeric threshold because it automatically adapts to each column's own scale and spread — a threshold appropriate for area_sqft (measured in thousands) would be meaningless for bedrooms (measured in single digits).

The results show bathrooms has the highest outlier count (400 values), followed by house_price_inr (288 values), while log_house_price shows far fewer outliers (63 values) despite representing essentially the same underlying price information. This large drop-off is a concrete, quantitative demonstration of why the log transformation from Section 8.3 is valuable: by compressing extreme high prices, it also brings many previously 'outlier' properties back within the normal statistical range, meaning fewer of them will disproportionately influence models trained on the log scale.

10. Feature Engineering

10.1. Price per Square Foot (exploratory only)

Code

```
df["price_per_sqft"] = df["house_price_inr"] / df["area_sqft"]

# Removed before modeling:
df = df.drop(columns=["price_per_sqft"])
```

Advantages:

- Useful, low-cost exploratory signal during EDA.
- Explicitly removing it protects the model from an obvious form of target leakage.

Interpretation:

This feature divides each property's price by its area to produce a per-square-foot pricing figure, a metric commonly used in real estate to compare value across differently sized properties. It is created here purely as an exploratory tool, to help understand pricing patterns during analysis rather than as an input the model should learn from.

Crucially, this column is dropped again later, before the data is split into features and target (Section 11). This is because price_per_sqft is calculated directly using house_price_inr — the very value the model is trying to predict. If left in the feature set, the model could essentially reverse-engineer the answer (price = price_per_sqft × area), which would produce misleadingly perfect results that would not hold up on genuinely new properties where the true price is unknown. Removing it is a deliberate, correct step to prevent this specific and fairly obvious form of data leakage.

10.2. Total Rooms

Code

```
df["total_rooms"] = df["bedrooms"] + df["bathrooms"]
```

Advantages:

- Reduces two correlated features into one simpler, potentially stronger signal.
- Cheap to compute and easy to interpret for non-technical stakeholders.

Interpretation:

This feature simply adds bedrooms and bathrooms together to create a single combined 'total rooms' figure. The reasoning is that bedroom and bathroom counts individually only show weak correlation with price (0.054 and 0.050 respectively, per Section 8.4), but their sum may capture a more general notion of property size and livability that either count alone does not fully express.

Engineering this feature is a low-risk, low-cost step: it requires no external data, cannot introduce missing values (since both source columns are complete), and gives the model — particularly the linear-based models — a more

direct, single-column signal for overall room count, rather than requiring the model to learn on its own that bedrooms and bathrooms should be considered together.

10.3. Property Age Category

Code

```
def age_category(age):
    if age <= 5:
        return "New"
    elif age <= 15:
        return "Moderate"
    else:
        return "Old"

df["property_age_category"] = df["property_age"].apply(age_category)
```

Advantages:

- Turns a continuous variable into interpretable bands that are easy to explain to non-technical stakeholders.
- Can help tree-based and linear models alike pick up threshold effects in age-related pricing.

Interpretation:

This custom function groups the continuous property_age column into three categories — New (0–5 years), Moderate (6–15 years), and Old (over 15 years) — using apply() to run the function across every row. The underlying assumption is that the effect of age on price may not be a smooth straight line; for example, a brand-new property might carry a premium that fades quickly in the first few years and then levels off, a pattern a single linear coefficient on raw age cannot represent well.

By converting age into these bands, and later one-hot encoding them alongside the other categorical features (Section 12.2), the model gains the flexibility to assign a different, independent effect to each age group rather than being forced to assume the price changes at a constant rate for every additional year of age. This is a common, practical feature engineering technique for capturing non-linear, threshold-like effects in linear models, and it also produces a feature that is immediately understandable to a non-technical audience.

10.4. Floor Ratio

Code

```
df["floor_ratio"] = df["floor_number"] / df["total_floors"]
```

Advantages:

- Normalizes floor position across buildings of very different heights.
- Provides a single, comparable engineered feature instead of relying on the model to learn the interaction between two separate columns.

Interpretation:

This feature divides a property's floor number by the total number of floors in its building, producing a ratio between 0 and 1 that represents relative vertical position — for example, a value close to 1 means the unit is near the top of its building, regardless of whether that building has 6 floors or 40.

Without this engineered ratio, the raw floor_number column alone is ambiguous: the 5th floor is near the top of a small building but near the bottom of a high-rise, and these two situations may have very different implications for price (such as views, noise, or perceived exclusivity). floor_ratio resolves this ambiguity in a single column, giving the model a normalized, directly comparable measure of building position across all properties rather than requiring it to infer the interaction between floor_number and total_floors on its own.

11. Train/Test Split

11.1. Splitting Features and Target

Code

```
X = df.drop(columns=["house_price_inr"])
y = df["house_price_inr"]

y_log = np.log1p(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_log, test_size=0.20, random_state=42
)
```

OUTPUT

```
X_train Shape: (8000, 20)
X_test Shape: (2000, 20)
y_train Shape: (8000,)
y_test Shape: (2000,)
```

Advantages:

- Guards against overly optimistic performance estimates caused by evaluating on training data.
- A fixed random_state makes the split, and therefore all downstream results, fully reproducible.
- An 80/20 split is a widely used balance between having enough data to train on and enough to evaluate reliably.

Interpretation:

This step first separates the dataset into X (every column except the target, house_price_inr) and y (the target itself), then creates a log-transformed version of the target, y_log, consistent with the transformation decided on during EDA. train_test_split() then randomly divides both X and y_log into a training portion (80%) and a testing portion (20%), using test_size=0.20 to set that ratio and random_state=42 to make the random split reproducible — anyone re-running this exact code will get the same 8,000/2,000 split every time.

This split exists to simulate how the model will behave on properties it has never seen: the model is trained only on X_train/y_train, and every performance metric reported later in the notebook is calculated on X_test/y_test, data the

model had no opportunity to memorize. The confirmed shapes — 8,000 training rows and 2,000 testing rows, each with 20 columns — verify the split executed correctly and that no rows were lost or duplicated relative to the original 10,000-row dataset. This step is the foundation that makes every later RMSE, MAE, and R^2 result meaningful as an estimate of real-world performance rather than a measure of how well the model memorized its own training data.

12. Preprocessing Pipeline

12.1. Numerical Feature Pipeline

Code

```
numerical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

Advantages:

- Median imputation is far less distorted by extreme values than mean imputation.
- Scaling prevents features with naturally larger numeric ranges (like area) from dominating models that are sensitive to feature magnitude, such as Linear/Ridge/Lasso Regression.

Interpretation:

This Pipeline chains two transformation steps that will always be applied in the same order: first SimpleImputer(strategy='median'), which fills any missing numeric value with that column's median, and then StandardScaler(), which rescales every numeric column to have a mean of 0 and a standard deviation of 1. Wrapping both steps in a single Pipeline object means they can be called together as one unit rather than as two separate, easy-to-mismatch operations.

Median imputation is chosen specifically over mean imputation because several numeric columns (area_sqft, house_price_inr) were shown to be skewed in Section 8; the median is far less sensitive to extreme values than the mean, so it produces a more representative 'typical value' fill-in. Scaling matters because the numeric columns in this dataset span very different ranges — area_sqft is in the thousands while bedrooms is a single digit — and without scaling, models like Linear, Ridge, and Lasso Regression (which are sensitive to the magnitude of each feature) would implicitly treat area_sqft as far more important simply because its raw numbers are larger, regardless of its actual predictive value.

12.2. Categorical Feature Pipeline

Code

```
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

Advantages:

- Most-frequent imputation is a simple, sensible default for categorical data.
- handle_unknown='ignore' prevents the pipeline from crashing if the test data contains a category not seen during training — an important safeguard for production use.

Interpretation:

This second Pipeline mirrors the structure of the numerical one but uses transformations appropriate for text-based categorical columns. SimpleImputer(strategy='most_frequent') fills any missing category (in location, property_type, or furnishing_status) with whichever category appears most often in that column, since a median or mean has no meaning for non-numeric data. OneHotEncoder then converts each category into a set of binary (0/1) columns, because machine learning algorithms require numeric input and cannot directly process text labels like 'Chennai' or 'Apartment'.

The handle_unknown='ignore' setting is a specific safeguard: if the test set (or, in a real deployment, new incoming data) contains a category value that was never seen during training, the encoder will not raise an error and halt the pipeline — it will simply encode that value as all zeros across the known categories. This is a small but important detail for production reliability, since an unhandled unseen category would otherwise crash the entire prediction pipeline.

12.3. Combining Both with ColumnTransformer

Code

```
preprocessor = ColumnTransformer(transformers=[
    ("numerical", numerical_pipeline, numerical_features),
    ("categorical", categorical_pipeline, categorical_features)
])
```

Advantages:

- Eliminates the risk of manually mismatching preprocessing steps between training and test data.
- Bundling preprocessing into one object means the exact same transformations are guaranteed to be applied consistently every time the pipeline is used, including in production.

Interpretation:

ColumnTransformer is used to route each column to the correct pipeline automatically: numerical_features are sent through numerical_pipeline (imputation + scaling), while categorical_features are sent through categorical_pipeline (imputation + one-hot encoding), and the results of both are combined into a single, unified feature matrix. Without ColumnTransformer, this routing would have to be done manually with custom code, increasing the risk of applying the wrong transformation to the wrong column type.

This preprocessor object is later embedded as the first step inside every model's own Pipeline (Section 13), which guarantees that training data and test data — and, in the future, any brand-new property submitted for prediction —

are always transformed in exactly the same way, using the exact same fitted imputers, scaler, and encoder learned from the training set. This consistency is essential for producing trustworthy, reproducible predictions and is one of the primary reasons scikit-learn pipelines are considered a best practice for production-ready machine learning code.

13. Model Training

Each model below is wrapped in a scikit-learn Pipeline that chains the shared preprocessor with the estimator, so raw data can be passed directly to .fit() and .predict() without repeating preprocessing code.

13.1. Model Evaluation Helper Function

Code

```
def evaluate_model(model, X_test, y_test):
    y_pred_log = model.predict(X_test)
    y_pred = np.expm1(y_pred_log)
    y_actual = np.expm1(y_test)

    rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
    mae = mean_absolute_error(y_actual, y_pred)
    r2 = r2_score(y_actual, y_pred)
    return rmse, mae, r2
```

Advantages:

- A single reusable function avoids duplicating evaluation logic for each of the five models.
- Reporting metrics in real price terms (rather than log units) makes results meaningful to non-technical audiences.

Interpretation:

This function defines a standard, reusable evaluation procedure that every model in the notebook follows: generate predictions on the test set (which come out in log-price units, since the models were trained on y_log), convert both the predictions and the true test values back to real INR prices using np.expm1() (the exact mathematical inverse of the earlier np.log1p() transformation), and then calculate RMSE, MAE, and R^2 on those real-price values.

Defining this logic once as a function, rather than repeating it for each of the five models, keeps the notebook consistent and reduces the chance of a calculation mistake in any individual model's evaluation. Converting back to real INR before computing the metrics is also an important interpretability choice: an error reported in log-price units (a small fraction) would be difficult for a business stakeholder to act on, whereas an error reported in actual rupees is immediately meaningful.

13.2. Linear Regression

Code

```
linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
```

```
    ("model", LinearRegression())
])

linear_model.fit(X_train, y_train)
```

```
OUTPUT
Training Time: 0.17 seconds
RMSE: 3.90e-08   MAE: 1.45e-08   R² Score: 1.0
```

Advantages:

- Extremely fast to train, making it a good first baseline to compare more complex models against.
- Coefficients are directly interpretable, showing which features push price up or down.
- Requires no hyperparameter tuning.

Interpretation:

Linear Regression is trained first as the project's baseline model — it assumes house price can be estimated as a weighted sum of the input features, with no built-in complexity beyond that straight-line relationship. It is included specifically because it is fast, requires no tuning, and produces coefficients that are easy to explain, making it a natural reference point against which the more complex models (Random Forest, XGBoost) can be judged.

The reported results — an RMSE of essentially zero and an R^2 of exactly 1.0 — are, on the surface, a 'perfect' fit. In practice, a real-world price prediction model achieving literally zero error on unseen test data is not a realistic outcome and is a strong signal of a data leakage problem rather than an authentically flawless model. As detailed fully in Section 15.3, this happens because a log-transformed copy of the target price was inadvertently left in the input features. This result should therefore be read as a diagnostic finding rather than a genuine success, and it is one of the most important interpretive takeaways in this entire documentation.

13.3. Ridge Regression (L2 Regularization)

Code

```
ridge_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RidgeCV(alphas=[0.01, 0.1, 1, 10, 100]))
])

ridge_model.fit(X_train, y_train)
```

```
OUTPUT
Training Time: 0.05 seconds
Best Ridge Alpha: 0.01
RMSE: 62.78   MAE: 43.91   R² Score: ~1.0
```

Advantages:

- Reduces the risk of overfitting compared to plain Linear Regression.
- Automatic alpha selection removes manual guesswork in choosing the regularization strength.
- Still fast to train and keeps model coefficients interpretable.

Interpretation:

Ridge Regression is trained next to test whether adding regularization — a penalty that discourages the model from assigning very large weights to any single feature — changes performance meaningfully compared to plain Linear Regression. RidgeCV automatically tries five candidate penalty strengths (0.01 through 100) and selects whichever performs best under cross-validation, removing the need to manually guess a good value.

The selected alpha of 0.01 is the smallest candidate tested, meaning the cross-validation procedure found that very little regularization was actually beneficial for this data — a model with almost no penalty performed about as well as one with more. The RMSE (62.78) and MAE (43.91), while technically larger than Linear Regression's near-zero error, are still extremely small relative to house prices that range into the tens of millions of rupees, and the R^2 remains effectively 1.0. This is consistent with the same root cause discussed for Linear Regression: the leaking feature is still present, and Ridge's mild penalty was not strong enough to meaningfully suppress its outsized influence.

13.4. Lasso Regression (L1 Regularization)

Code

```
lasso_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LassoCV(alphas=[0.0001, 0.001, 0.01, 0.1, 1], max_iter=10000))
])

lasso_model.fit(X_train, y_train)
```

OUTPUT

```
Training Time: 0.09 seconds
Best Lasso Alpha: 0.0001
Total Features: 38   Zeroed-Out Features: 37
```

Advantages:

- Simplifies the model by identifying and discarding features that add little predictive value.
- The count of zeroed-out coefficients is itself a useful diagnostic — in this case it flagged that the model was leaning on essentially one dominant feature (see Section 15).

Interpretation:

Lasso Regression uses a different type of penalty (L1) than Ridge, one with the distinctive property of being able to shrink a feature's coefficient all the way down to exactly zero rather than just making it smaller. This effectively performs automatic feature selection: any feature Lasso zeroes out is being treated as contributing nothing to the prediction. LassoCV again automatically searches across several candidate penalty strengths to find the best-performing one via cross-validation.

The diagnostic output here is especially telling: out of 38 total input features (after one-hot encoding expands the categorical columns), Lasso set 37 of them to exactly zero, keeping only a single feature with any influence at all. A model that discards 97% of its available features and still achieves near-perfect accuracy is not behaving the way a healthy, well-generalizing model should on real-world tabular data — it is a strong quantitative confirmation that one feature is carrying almost the entire prediction, which, as shown by the feature-importance analysis in Section 15.3, is the leaking log_house_price column.

13.5. Random Forest Regressor

Code

```
random_forest_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=200, max_depth=10, random_state=42, n_jobs=-1))
])

random_forest_model.fit(X_train, y_train)
```

OUTPUT

```
Random Forest Training Time: 9.96 seconds
RMSE: 477,493.33   MAE: 21,082.15   R² Score: 0.9984
```

Advantages:

- Handles non-linear relationships and feature interactions without manual feature crossing.
- Less sensitive to feature scaling and outliers than linear models.
- n_jobs=-1 parallelizes tree-building across all available CPU cores, speeding up training.

Interpretation:

Random Forest is introduced as a fundamentally different type of model: rather than fitting a single straight-line equation, it builds 200 individual decision trees (n_estimators=200), each trained on a random subset of the data and features, and averages their predictions together. max_depth=10 limits how deep each tree can grow, which helps prevent any individual tree from memorizing the training data too closely. This ensemble approach allows the model to capture non-linear relationships and interactions between features (for example, how the effect of area on price might differ by location) that a linear model structurally cannot represent.

The results show a meaningfully different pattern from the three linear-based models: RMSE (477,493) and MAE (21,082) are now non-trivial values in real rupee terms, and R^2 has dropped slightly to 0.9984 from the near-exact 1.0 seen previously. This is actually informative rather than a weakness — it indicates that Random Forest is not relying on the leaking feature quite as absolutely as the linear models did (its feature importance analysis in Section 15.3 will quantify this further), producing errors that, while still unrealistically small for a genuine real-estate pricing problem, are closer to what a model influenced by, but not entirely dependent on, the leaked feature would produce.

13.6. XGBoost Regressor

Code

```
xgboost_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", XGBRegressor(
        n_estimators=500, learning_rate=0.05, max_depth=4,
        subsample=0.8, colsample_bytree=0.8, random_state=42,
        objective="reg:squarederror"))
])

xgboost_model.fit(X_train, y_train)
```

OUTPUT

```
XGBoost Training Time: 1.61 seconds
RMSE: 860,221.93   MAE: 177,654.14   R² Score: 0.9949
```

Advantages:

- subsample and colsample_bytree introduce randomness that helps prevent overfitting, similar in spirit to Random Forest's bagging.
- A low learning_rate (0.05) combined with many estimators (500) allows the model to learn gradually and generalize better than a few large, aggressive steps.
- Typically achieves strong accuracy while training much faster than an equivalently sized Random Forest.

Interpretation:

XGBoost takes a different ensemble strategy than Random Forest: instead of building 500 independent trees and averaging them, it builds trees sequentially, where each new tree is specifically trained to correct the errors made by the trees built before it (gradient boosting). The configuration here uses a deliberately small learning_rate (0.05) paired with a larger number of estimators (500), which is a well-established strategy for learning gradually and avoiding the kind of large, aggressive updates that lead to overfitting; subsample=0.8 and colsample_bytree=0.8 add further randomness by training each tree on only 80% of the rows and 80% of the columns, respectively, which also helps generalization.

The results are the 'weakest' of the five models by RMSE and MAE, and this is presented here as a meaningful, positive finding rather than a shortcoming: XGBoost's RMSE (860,222) and MAE (177,654) are the largest among the five, and its R^2 (0.9949) is the lowest, which — combined with its feature importance breakdown in Section 15.3 showing a more balanced reliance on area_sqft and location alongside the leaking feature — suggests XGBoost is the model least dependent on the single problematic column. In a corrected version of this project with the leakage removed, XGBoost's architecture (gradient boosting with regularized, shallow trees) would likely make it one of the strongest candidates for a genuinely reliable final model.

14. Understanding the Evaluation Metrics

RMSE (Root Mean Squared Error)

RMSE measures the typical prediction error in the same units as the target (INR), calculated by squaring each individual error, averaging those squared errors, and then taking the square root to bring the result back to the original price scale. Because errors are squared before averaging, RMSE penalizes large individual errors far more heavily than small ones — a single prediction that is off by a huge amount will increase RMSE much more than several predictions that are each off by a small amount. Lower RMSE indicates better performance, and it is the metric used in this project to automatically select the best model in Section 15.

MAE (Mean Absolute Error)

MAE measures the average absolute difference between predicted and actual prices, without squaring the errors first. This makes MAE more robust to a handful of very large errors than RMSE, and it has a very direct, intuitive interpretation: 'on average, the model's predictions are off by ₹X.' Comparing RMSE and MAE for the same model can also be diagnostic — when RMSE is much larger than MAE, it typically indicates the model is making a small number of unusually large errors rather than being consistently a little bit off across the board.

R^2 Score (Coefficient of Determination)

R^2 represents the proportion of variance in house price that the model is able to explain, on a scale where 1.0 means the model's predictions perfectly match the actual values, 0 means the model performs no better than simply always predicting the average price, and negative values mean the model performs worse than that naive average-based guess. While higher R^2 is generally desirable, this project is a useful reminder that an R^2 this close to 1.0 across every single model tested (Section 13) is not, on its own, proof of a good model — it can equally be a symptom of data leakage, as the detailed interpretation in Section 15.3 explains, and any R^2 this extreme on a real-world, noisy target like property price should prompt further investigation rather than immediate celebration.

15. Model Comparison & Final Model Selection

15.1. Comparison Table

Code

```
model_comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression",
              "Random Forest", "XGBoost"],
    "RMSE": [linear_rmse, ridge_rmse, lasso_rmse, rf_rmse, xgb_rmse],
    "MAE": [linear_mae, ridge_mae, lasso_mae, rf_mae, xgb_mae],
    "R2 Score": [linear_r2, ridge_r2, lasso_r2, rf_r2, xgb_r2]
})
```

OUTPUT

```
Model               RMSE          MAE       R2 Score
Linear Regression   3.90e-08      1.45e-08   1.000000
Ridge Regression     6.28e+01      4.39e+01   1.000000
```

```
Lasso Regression     2.78e+03      1.57e+03   1.000000
Random Forest        4.77e+05      2.11e+04   0.998414
XGBoost              8.60e+05      1.78e+05   0.994852
```

Advantages:

- Enables a single, defensible, data-driven basis for model selection (lowest RMSE) rather than an intuition-based choice.
- Makes trade-offs visible at a glance — e.g., training time vs. accuracy vs. interpretability.

Interpretation:

This step gathers the RMSE, MAE, and R^2 values calculated separately for each of the five models into one consolidated table, making it possible to compare them fairly, side by side, on identical evaluation criteria and the same held-out test set.

Reading the table as a whole reveals a clear pattern: the three linear-based models (Linear, Ridge, Lasso) report dramatically smaller errors and higher R^2 than the two tree-based models (Random Forest, XGBoost), even though tree-based models are generally at least as capable, and often more capable, of capturing complex relationships in structured data. This pattern — where the algorithmically simpler models substantially outperform the more sophisticated ones — is itself a red flag rather than a straightforward win for linear regression; it is explained in full in Section 15.3 and is the reason this documentation treats the 'best model' selection below as provisional rather than final.

15.2. Selecting the Best Model

Code

```
best_model_index = model_comparison["RMSE"].idxmin()
best_model_name = model_comparison.loc[best_model_index, "Model"]
```

OUTPUT

```
Best Model: Linear Regression
```

Advantages:

- Removes manual bias from model selection.
- Easy to re-run if new models are added to the comparison in the future.

Interpretation:

idxmin() locates the row in the comparison table with the lowest RMSE value, and that row's model name is then pulled out and stored as best_model_name. This automates model selection as an objective, reproducible rule (lowest error wins) rather than a subjective judgment call, and the same code can be re-run unchanged if additional models are added to the comparison in the future.

Applying this rule to the current results selects Linear Regression as the 'best' model. Taken at face value, this result is misleading: as established across Sections 13 and 15.3, Linear Regression's apparent superiority is a direct

consequence of it relying most completely on a leaked feature, not evidence that a simple linear relationship genuinely describes house pricing better than a gradient-boosted ensemble. This is precisely why an automated selection rule, however logical in principle, still requires a human reviewing the underlying features and results before the selected model is trusted or deployed.

15.3 Detailed Interpretation: Feature Leakage

During EDA (Section 8.3), a log-transformed copy of the target variable was added directly to the working DataFrame using the line df['log_house_price'] = np.log1p(df['house_price_inr']). Later, when features were separated from the target for modeling (Section 11.1), the code only dropped house_price_inr from X — the newly created log_house_price column was never removed, and so it remained available to every model as an input feature throughout Section 13.

Because log_house_price is mathematically almost identical to y_log, the log-transformed value every model was actually trained to predict, each model effectively had indirect access to the answer during both training and testing. This single oversight is the root cause of the unusually perfect results reported throughout Section 13: an R^2 of exactly 1.0 for the linear-based models, and R^2 values of 0.998 and 0.995 for Random Forest and XGBoost respectively. This is not a case of the models being unusually well-designed — it is a textbook example of target leakage, one of the most common and most consequential mistakes in applied machine learning.

The notebook's own feature-importance outputs confirm this directly and quantitatively rather than leaving it as speculation. Random Forest's importance ranking (Section 13.5) assigns log_house_price an importance score of 0.9999805 — meaning essentially 100% of that model's decision-making is driven by this single feature, with every genuine property attribute (area, location, age, and so on) contributing a negligible remainder. XGBoost's importance ranking (Section 13.6) assigns log_house_price a somewhat lower but still dominant importance of 0.65, with area_sqft as the next most influential feature at 0.152 — indicating XGBoost, while still affected, distributed more of its decision-making across genuine property characteristics than the other four models did.

Why this matters: In a real-world deployment, log_house_price would not be available at prediction time, because it is derived directly from the very value the model is being asked to predict for a new, unpriced property. A model that appears to perform perfectly in this notebook would fail immediately in production, since the input it has learned to depend on most heavily simply would not exist for a new listing.

Recommended fix: Drop log_house_price from X alongside house_price_inr in Section 11.1, before the train/test split, and re-run the full model training and comparison in Section 13 and 15.1. The resulting metrics — particularly XGBoost's, which already relied comparatively more on genuine features like area_sqft even with the leak present — would give a realistic, trustworthy picture of how accurately this dataset can actually predict house prices, and the automated best-model selection in Section 15.2 could then be relied upon without this caveat.

16. Cross-Validation & Hyperparameter Tuning

16.1. Cross-Validation on XGBoost

Code

```
cv_scores = cross_val_score(
    xgboost_model, X_train, y_train,
    cv=5, scoring="neg_root_mean_squared_error", n_jobs=-1
)
cv_rmse_scores = -cv_scores
```

OUTPUT

```
Mean CV RMSE: 0.0169
Standard Deviation: 0.0011
CV RMSE: 0.02 ± 0.00
```

Advantages:

- Reduces the risk that reported performance is an artifact of one particular data split.
- The standard deviation across folds shows how stable the model's performance is — a small value indicates consistent results.

Interpretation:

cross_val_score with cv=5 retrains and evaluates the XGBoost pipeline five separate times, each time using a different 80/20 split of the training data, rather than relying on the single train/test split used everywhere else in the notebook. Because scikit-learn's scoring convention treats higher values as better, RMSE (where lower is better) has to be requested as its negative form, neg_root_mean_squared_error, and then negated back (cv_rmse_scores = -cv_scores) to interpret it as a normal, positive error value.

The results — a mean RMSE of about 0.0169 with a standard deviation of only about 0.0011 — are reported in log-price units, since cross-validation is being run on the training pipeline that predicts y_log directly, before any expm1() conversion back to rupees. The small standard deviation relative to the mean indicates that XGBoost's performance is stable and consistent across five different random subsets of the training data, rather than being an artifact of one particularly favorable train/test split. This is a genuinely useful, leakage-independent finding: whatever the model is learning, it applies that learning consistently, which is a separate and valid measure of reliability from the (leakage-inflated) R^2 scores discussed in Section 15.3.

16.2. Randomized Hyperparameter Search (Random Forest)

Code

```
param_distributions = {
    "model__n_estimators": [100, 200, 300, 500],
    "model__max_depth": [5, 10, 15, 20, None],
    "model__min_samples_split": [2, 5, 10],
```

```
    "model__min_samples_leaf": [1, 2, 4]
}

random_search = RandomizedSearchCV(
    estimator=random_forest_model,
    param_distributions=param_distributions,
    n_iter=20, cv=3, scoring="neg_root_mean_squared_error",
    random_state=42, n_jobs=-1
)
```

Advantages:

- Scales far better than exhaustive grid search when the parameter space is large.
- Combined with cross-validation (cv=3), it evaluates each candidate combination robustly rather than on a single split.

Interpretation:

param_distributions defines a search space of Random Forest hyperparameters — the number of trees, maximum tree depth, and the minimum samples required to split or form a leaf node — that could plausibly improve the model. RandomizedSearchCV is used instead of an exhaustive grid search because testing every combination of these four parameters (4 × 5 × 3 × 3 = 180 combinations, each evaluated across multiple cross-validation folds) would be computationally expensive; instead, n_iter=20 tells it to sample and evaluate only 20 random combinations, using cv=3 (3-fold cross-validation) to score each one robustly.

It is important to note, for accuracy, that this search object is fully configured in the notebook but its .fit() method is never called in any subsequent cell. As a result, no tuned Random Forest model is actually produced or used anywhere later in the project — the final model comparison in Section 15.1 uses the originally configured Random Forest from Section 13.5 (with its manually chosen n_estimators=200, max_depth=10), not a hyperparameter-tuned version. Completing this step by calling random_search.fit(X_train, y_train) and using random_search.best_estimator_ downstream is one of the concrete recommendations in the Conclusion (Section 18).

17. Final Model, Visualization & Deployment

17.1. Residual Analysis

Code

```
final_residuals = actual_prices - final_predictions

plt.scatter(final_predictions, final_residuals, alpha=0.6)
plt.axhline(y=0, linestyle='--')
```

Advantages:

- A residual plot can reveal systematic bias that a single aggregate metric like RMSE would hide.

- Widely used, standard diagnostic for regression model quality.

Interpretation:

A residual is the difference between what actually happened and what the model predicted (actual price minus predicted price). Plotting residuals against predicted values, with a horizontal reference line at zero, is a standard regression diagnostic: a healthy, well-fitting model should show residuals scattered randomly and evenly around zero across the entire range of predicted values, with no visible pattern, funnel shape, or curve.

In this notebook, the residuals cluster extremely tightly around zero across the full price range, which at first glance looks like an ideal diagnostic result. However, this observation must be read together with Section 15.3: residuals this small and this consistently near-zero, for every price level, are consistent with a model that has near-perfect information about the answer (via the leaked feature) rather than with a model that has learned a genuinely difficult pricing relationship from noisy, real-world property characteristics. A residual plot from the corrected, leakage-free model would be expected to show a wider, more realistic scatter around zero.

17.2. Saving the Model with joblib

Code

```
joblib.dump(random_forest_model, "final_house_price_model.pkl")
# ... later in the notebook:
joblib.dump(xgboost_model, "final_house_price_model.pkl")

loaded_model = joblib.load("final_house_price_model.pkl")
```

OUTPUT

```
Final model loaded successfully.
```

Advantages:

- Saving the whole pipeline (not just the raw model) guarantees new data is preprocessed identically to the training data.
- .pkl files load almost instantly compared to re-running the full training process.

Interpretation:

joblib.dump() serializes an entire fitted Python object — in this case, the whole scikit-learn Pipeline, including its fitted preprocessor and trained estimator — into a single binary .pkl file on disk, so it can be reloaded later and used to generate predictions without repeating any of the training process. Saving the whole pipeline object, rather than just the raw model, is what guarantees any new data passed to it will be imputed, scaled, and encoded in exactly the same way the training data was.

In this notebook, the same filename, final_house_price_model.pkl, is used twice: first to save random_forest_model, and later to save xgboost_model. Because the second dump() call overwrites the first, the file that actually exists on disk after these two cells run contains the XGBoost pipeline, not the Random Forest pipeline, and — notably — not the Linear Regression model that Section 15.2 identified as the automatically selected 'best' model either. loaded_model, used throughout the remainder of the notebook, therefore refers to XGBoost specifically. This is flagged here as a workflow inconsistency worth correcting in a production setting: the model that is actually saved

and deployed should be an explicit, deliberate choice that matches the model selected through the evaluation process, not an artifact of which dump() call happened to run last.

17.3. Sample Predictions

Code

```
sample_X = X_test.head(5)
sample_y = y_test.head(5)

sample_predictions_log = loaded_model.predict(sample_X)
sample_predictions = np.expm1(sample_predictions_log)
sample_actual_prices = np.expm1(sample_y)
```

OUTPUT

```
  Actual Price (INR)  Predicted Price (INR)  Absolute Error
0           12675000               12631274           43726
1           33414000               33034424          379576
2            2771000                2745363           25637
```

Advantages:

- Gives stakeholders a tangible, human-readable demonstration rather than only abstract metrics.
- Confirms the saved/reloaded pipeline works correctly end-to-end, including the log-to-price conversion.

Interpretation:

This step pulls five individual properties from the held-out test set and passes them through the reloaded model exactly as they would be passed through in a real prediction scenario: generate a log-scale prediction, then convert both the prediction and the true value back to real INR using np.expm1(), and finally present them together in a simple comparison table with the absolute error calculated for each property.

This demonstration serves two purposes. First, it confirms end-to-end that the saved-and-reloaded pipeline works correctly — including the imputation, scaling, encoding, prediction, and inverse log transformation all functioning together as intended. Second, it gives a concrete, individually verifiable example of model performance that is easier for a non-technical reader to evaluate than an abstract RMSE or R^2 figure; readers can look at each row and judge for themselves whether an error like ₹43,726 on a ₹12,675,000 property feels reasonable, in a way that is more intuitive than interpreting a single aggregate statistic.

17.4. Final Visual Comparisons

Code

```
plt.plot(comparison_df.index, comparison_df['Actual Price'], marker='o', label='Actual
Price')
plt.plot(comparison_df.index, comparison_df['Predicted Price'], marker='o', linestyle='--',
label='Predicted Price')
```

```
sns.scatterplot(x=actual_prices, y=final_predictions, alpha=0.7)
plt.plot([min_value, max_value], [min_value, max_value], linestyle='--', label='Perfect
Prediction')
```

Advantages:

- The diagonal reference line gives an instant visual benchmark — points on the line are exactly correct, points off it show the size and direction of the error.
- Annotated line charts make individual prediction errors easy to spot for specific properties.

Interpretation:

Two complementary chart types are used to communicate final model accuracy. The line chart plots actual and predicted prices for the first 50 test properties side by side, with markers and annotated values, making it easy to trace individual properties and see exactly where predicted and actual prices diverge. The scatter plot instead plots every test property as a single point (actual price on one axis, predicted price on the other) and overlays a diagonal 'perfect prediction' reference line — any point that would fall exactly on this line represents a prediction that exactly matched the true price.

In both charts, predicted and actual values track each other almost exactly, with points sitting essentially on top of the reference diagonal. As with the residual plot in Section 17.1, this visual result should not be interpreted in isolation as evidence of an outstanding model — it is the visual expression of the same feature leakage discussed at length in Section 15.3. These charts remain useful primarily as a demonstration of what the visualization technique itself looks like, and they would become genuinely meaningful performance evidence once applied to a version of the model retrained without the leaking log_house_price feature.

18. Conclusion

This project demonstrates a complete, professionally structured regression workflow — from raw data to a saved, deployable model — covering data quality checks, exploratory analysis, feature engineering, pipeline-based preprocessing, multi-model training, evaluation, comparison, and cross-validation. Each design choice, from log-transforming the skewed target to using ColumnTransformer for consistent preprocessing, was made to directly address a characteristic observed in the data, and the workflow itself — independent of the specific numeric results — reflects sound, industry-standard practice.

At the same time, a detailed, code-grounded review of the results surfaced a significant and instructive issue: a log-transformed copy of the target variable was left in the model's input features, causing every one of the five trained models to report unrealistically high accuracy. This finding does not diminish the value of the workflow — identifying, explaining, and correcting exactly this kind of issue is itself a core, professional machine learning skill, and the diagnostic evidence for it (feature importance rankings, the pattern of linear models outperforming tree-based ones, and the traceable origin of the leaking column) is laid out in full in Section 15.3.

Recommendations for a production-ready version

- Remove log_house_price from the feature set (X) before training, and re-run the full model comparison to obtain realistic performance metrics.
- Re-select the final model based on the corrected comparison table — XGBoost, which already relied more heavily on genuine features like area_sqft, is a strong candidate.

- Explicitly save only the chosen final model (avoid overwriting the saved file with a different model than the one selected).
- Complete the hyperparameter search (RandomizedSearchCV) by calling .fit() and using the tuned model in the final comparison.
- Consider additional real-world features (e.g., neighborhood amenities, school proximity) to further improve genuine predictive power once leakage is corrected.